



**MATEMATICKO-FYZIKÁLNÍ
FAKULTA**
Univerzita Karlova

BAKALÁŘSKÁ PRÁCE

Stanislav Dochynets

Rozhraní pro plantri a geng

Informatický ústav Univerzity Karlovy

Vedoucí bakalářské práce: prof. Mgr. Zdeněk Dvořák, Ph.D.

Studijní program: Informatika

Studijní obor: Programování a vývoj software

Praha 2026

Prohlašuji, že jsem tuto bakalářskou práci vypracoval(a) samostatně a výhradně s použitím citovaných pramenů, literatury a dalších odborných zdrojů. Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona v platném znění, zejména skutečnost, že Univerzita Karlova má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona.

V dne

Podpis autora

Věnování: Rád bych poděkoval svému vedoucímu práce za odborné vedení, ochotu věnovat svůj čas konzultacím a za cenné připomínky a rady, které významně přispěly ke vzniku této práce.

Velké poděkování patří také mým rodičům, zejména mé mamince, za jejich trvalou podporu, trpělivost a povzbuzení po celou dobu studia.

Název práce: Rozhraní pro plantri a geng

Autor: Stanislav Dochynets

Ústav: Informatický ústav Univerzity Karlovy

Vedoucí bakalářské práce: prof. Mgr. Zdeněk Dvořák, Ph.D., Informatický ústav Univerzity Karlovy

Abstrakt: Tato práce představuje knihovnu PlanG, která zpřístupňuje generátory neizomorfních obecných a rovinných grafů `geng` a `plantri` prostřednictvím jednotného rozhraní v C++. Řeší tím obtížné propojení jejich nízkourovňových C rozhraní a interních datových struktur s moderním C++ kódem. Knihovna poskytuje typově bezpečnou konfiguraci, callbacky pro prořezávání, filtrování a zpracování výsledků a views nad interními daty bez zbytečného kopírování. Oba backendy podporují vybrané koncepty Boost Graph Library. Generátor `geng` byl navíc rozšířen o kanonické generování grafů s barevnými třídami a zakotvenými vrcholy. Měření ukazují srovnatelný výkon s původními generátory při čistém generování a malou režii callbackové vrstvy. Knihovna tak usnadňuje experimentální zkoumání grafů, testování algoritmů a hledání protipříkladů.

Klíčová slova: generování grafů, obecné grafy, rovinné grafy, barvení grafů, `geng`, `plantri`

Title: Interface for plantri and geng

Author: Stanislav Dochynets

Institute: Computer Science Institute of Charles University

Supervisor: prof. Mgr. Zdeněk Dvořák, Ph.D., Computer Science Institute of Charles University

Abstract: This thesis presents PlanG, a library that provides access to the non-isomorphic graph generators `geng` and `plantri` for general and planar graphs through a unified C++ interface. It addresses the challenge of integrating their low-level C APIs and internal data structures with modern C++ code. The library offers type-safe configuration, callbacks for pruning, filtering, and result processing, as well as views over internal data without unnecessary copying. Both backends support selected concepts of the Boost Graph Library. In addition, the `geng` generator has been extended with canonical graph generation using vertex color classes and rooted vertices. Performance evaluation shows that PlanG achieves performance comparable to the original generators during pure graph generation while introducing only a small overhead for the callback layer. Library therefore facilitates experimental graph exploration, algorithm testing, and the search for counterexamples.

Keywords: graph generation, general graphs, planar graphs, graph coloring, `geng`, `plantri`

Obsah

Úvod	3
1 Teoretický úvod	5
1.1 Počítačem podporovaná teorie grafů	5
1.2 Generování grafů	5
1.3 Izomorfismus grafů	6
1.4 Kanonické reprezentace	7
1.5 Backtracking a prořezávání	7
1.6 Praktické komplikace generování grafů	8
2 Generátory grafů geng a plantri	9
2.1 Generátor geng	9
2.2 Generátor plantri	10
2.3 Možnosti rozšíření pomocí callbacků	11
2.4 Omezení původních rozhraní	11
3 Návrh rozhraní	13
3.1 Cíle návrhu	13
3.2 Jednotné rozhraní pro více generátorů	14
3.3 Callbacky pro filtrování a zpracování grafů	14
3.4 View na graf bez kopírování dat	15
3.5 Integrace s Boost Graph Library	15
3.6 Podpora bohatších struktur	15
4 Implementace	17
4.1 Architektura knihovny	17
4.2 Vrstva Generator a backendy	18
4.3 Propojení s geng	18
4.4 Propojení s plantri	19
4.5 Integrace s Boost Graph Library	20
4.6 Obarvené a zakotvené vrcholy v geng	20

5	Testování a vyhodnocení	23
5.1	Funkční testování	23
5.2	Metodika měření	23
5.3	Čisté generování	24
5.4	Callbacks a srovnání s pluginem	25
5.5	Cena bohatších struktur	26
5.6	Shrnutí	26
	Závěr	27
	Seznam použité literatury	29
A	Použití knihovny PlanG	31
A.1	Sestavení projektu	31
A.2	Základní program	32
A.3	Callbacks	33
A.4	Práce s grafem	33
A.5	Použití algoritmů Boost Graph Library	35
A.6	Obarvené a zakotvené vrcholy v backendu geng	35
A.7	Dostupné ukázky a benchmarky	37

Úvod

Generování grafů je důležitým nástrojem experimentální teorie grafů a testování grafových algoritmů. Umožňuje systematicky procházet grafy splňující zadané podmínky, ověřovat na nich hypotézy, hledat protipříklady a porovnávat chování algoritmů. Efektivní generování je však obtížné, protože počet grafů rychle roste a stejný graf může vzniknout v mnoha izomorfních kopiích.

Pro tyto úlohy existují specializované generátory, například `geng` z balíku `nauty` pro obecné neorientované grafy a `plantri` pro rovinné grafy. Oba nástroje jsou velmi efektivní a lze je snadno spustit z příkazové řádky. Jejich zapojení do vlastního programu je však složitější. Filtrování grafů během generování obvykle vyžaduje psaní nízkoúrovňových pluginů a znalost interních datových struktur jednotlivých generátorů.

V této práci navrhujeme a implementujeme knihovnu `PlanG`, která zpřístupňuje oba generátory prostřednictvím jednotného rozhraní v jazyce C++. Uživatel může pomocí typově bezpečných metod nastavit parametry generování a registrovat callbacky pro prořezávání, filtrování a zpracování výsledků. Vygenerované grafy jsou callbackům předávány pomocí `views` přímo nad interními daty generátorů, takže nevzniká kopie celého grafu. Oba backendy navíc poskytují grafy kompatibilní s vybranými koncepty `Boost Graph Library`.

Součástí práce je také rozšíření generátoru `geng` o barevné třídy a zakotvené vrcholy. Barvy jsou zahrnuty přímo do testů izomorfismu a kanonického rozhodování, což umožňuje generovat grafy s dodatečnou strukturou bez následného vytváření všech obarvených hotových grafů.

Výsledná knihovna byla ověřena funkčními testy a porovnána s původními generátory. Měření ukazují, že při čistém generování dosahuje `PlanG` srovnatelného výkonu jako nativní programy.

První kapitola shrnuje teoretické základy generování grafů. Druhá kapitola popisuje generátory `geng` a `plantri` a omezení jejich původních rozhraní. Třetí kapitola popisuje návrh knihovny `PlanG`. Čtvrtá kapitola se zabývá implementací rozhraní a úpravami původních generátorů. Pátá kapitola se věnuje funkčnímu testování a vyhodnocení výkonu.

Deklarace použití AI. Při přípravě této práce jsem používal nástroje generativní umělé inteligence pouze jako podpůrný prostředek při jazykových úpravách, formulaci textu a kontrole jeho srozumitelnosti. Odborný obsah práce, implementace projektu i výsledné závěry jsou výsledkem vlastní práce autora

Kapitola 1

Teoretický úvod

1.1 Počítačem podporovaná teorie grafů

Počítače se v teorii grafů nepoužívají pouze k numerickým výpočtům, ale také jako nástroj pro objevování a ověřování matematických tvrzení. V počítačem podporované teorii grafů můžeme například prohledávat grafy dané velikosti, vybírat z nich grafy se zadanými vlastnostmi a testovat na nich navržená tvrzení.

Typickou úlohou je hledání protipříkladů k tvrzením tvaru: každý graf s vlastností P_1 má také vlastnost P_2 . Pokud takové tvrzení neplatí, chceme najít graf, který splňuje P_1 , ale nesplňuje P_2 . Způsob generování přitom výrazně ovlivňuje efektivitu výpočtu. Můžeme nejprve vygenerovat všechny grafy a následně testovat obě vlastnosti, nebo už během generování omezit výstup na grafy splňující P_1 . [1, Section 2].

Úplné generování můžeme využít také při hledání nejmenšího grafu se zadanou vlastností. Prozkoumáme-li všechny grafy až do určitého počtu vrcholů a žádný z nich danou vlastnost nesplňuje, získáme dolní mez velikosti hledaného grafu. Nalezený graf naopak poskytuje horní mez.

Takový postup sám o sobě nenahrazuje matematický důkaz obecného tvrzení. Je však užitečný ve fázi hledání hypotéz a protipříkladů. Pokud nemůžeme tvrzení vyvrátit na malých grafech, může nám počítač pomoci odhadnout jeho správnou formulaci a nasměrovat hledání důkazu. Pokud naopak najdeme malý protipříklad, může nám pomoci pochopit, proč tvrzení neplatí.

1.2 Generování grafů

Generováním grafů budeme rozumět postup, při kterém algoritmus vytváří grafy z předem určené třídy. V této práci se zaměříme na úplné generování, jehož cílem je vytvořit všechny grafy dané velikosti splňující zadané podmínky.

V nejjednodušším případě bychom pro pevný počet vrcholů mohli projít všechny možné hrany mezi těmito vrcholy. Pro graf na n vrcholech existuje $\binom{n}{2}$ možných hran, a tedy $2^{\binom{n}{2}}$ označených neorientovaných grafů. Toto číslo roste velmi rychle, takže přímé procházení všech možností použijeme jen pro malé hodnoty n .

V mnoha případech nám navíc nezáleží na konkrétních jménech vrcholů. Dva grafy, které se liší pouze přejmenováním vrcholů, představují z hlediska mnoha vlastností grafů stejný případ. Pokud bychom generovali všechny označené grafy, jeden graf bychom zpracovávali v mnoha různých případech. Praktické generátory se proto snaží vytvořit pouze jeden reprezentant z každé izomorfní třídy.

Rychlost generování můžeme dále zlepšit omezením požadované třídy. Můžeme například předepsat počet hran, stupně vrcholů, souvislost nebo rovinnost. Čím více použitelných podmínek generátor zná předem, tím větší část prostoru může vynechat. Specializovaný generátor určité třídy proto může být výrazně rychlejší než obecný generátor následovaný filtrováním výsledků.

Praktické generátory obvykle nevytvářejí každý graf nezávisle od začátku. Graf budují postupně, například přidáváním vrcholů nebo hran, a průběžně kontrolují, zda rozpracovaný graf ještě může vést k požadovanému výsledku [1, Section 2.2]. Pokud to možné není, další rozšiřování ukončí. Tím mohou včas vyřadit nepotřebné větve a zmenšit prohledávaný prostor.

Vzhledem k velkému počtu výsledků se vygenerované grafy obvykle neuchovávají všechny v paměti. Generátor je průběžně zapisuje do souboru nebo na standardní výstup, odkud je může zpracovávat další program. V paměti tak stačí uchovávat pouze aktuální graf.

1.3 Izomorfismus grafů

Dva grafy G a H nazveme izomorfní, pokud existuje bijekce mezi jejich vrcholy, která zachovává sousednost. Jinými slovy, grafy mají stejnou strukturu a liší se pouze pojmenováním vrcholů. Při generování grafů je izomorfismus zásadní, protože nechceme opakovaně zpracovávat různé popisy téhož grafu.

Izomorfismus grafu se sebou samým nazýváme automorfismus. Automorfismy mohou rozdělit vrcholy grafu do orbit. Dva vrcholy leží ve stejné orbitě, pokud existuje automorfismus, který jeden zobrazí na druhý. Při generování proto často stačí vyzkoušet jedno rozšíření z každé orbity, protože odpovídající rozšíření ostatních prvků by vedla k izomorfním výsledkům [1, Section 2.2]

Pokud grafy obsahují další strukturu, například barvy vrcholů nebo vybrané zakotvené vrcholy, musí izomorfismus respektovat i tyto informace. U obarvených grafů tedy můžeme zaměňovat pouze vrcholy se stejnou barvou. Tato myšlenka je

pro nás důležitá, protože v této práci rozšiřujeme generátor geng o barevné třídy a zakotvené vrcholy, které musí izomorfismus v procesu generování respektovat.

Testování izomorfismu může být výpočetně náročné a během generování se provádí velmi často. Efektivní generátory proto využívají specializované algoritmy, automorfismy a kanonické značení, které jim umožňují odmítat izomorfní kopie již během generování.

1.4 Kanonické reprezentace

Kanonická reprezentace přiřazuje každému grafu jednoznačný popis, který nezávisí na původním označení jeho vrcholů. Pro kanonickou reprezentaci $\text{can}(G)$ požadujeme, aby pro každé dva grafy G a H platilo

$$G \cong H \iff \text{can}(G) = \text{can}(H).$$

Izomorfní grafy tedy získají stejnou reprezentaci, zatímco neizomorfní grafy musí získat reprezentace různé.

Kanonickou reprezentaci můžeme teoreticky získat například tak, že vyzkoušíme všechna přeznačení vrcholů a vybereme lexikograficky nejmenší matici sousednosti. Takový postup by však vyžadoval prozkoumat až $n!$ přeznačení. Praktické algoritmy proto využívají dělení vrcholů do skupin, automorfismy a další postupy, které umožňují většinu možností vynechat. Nástrojem pro výpočet kanonického značení a automorfismů grafů je například knihovna nauty [2].

Jedním z možných způsobů odstranění izomorfních kopií je ukládat kanonické reprezentace již vytvořených grafů. U každého nového grafu potom zkontrolujeme, zda se jeho reprezentace v uložené množině již nachází. Tento přístup je jednoduchý, ale při velkém počtu grafů vyžaduje značné množství paměti.

Efektivní generátory proto často zahrnují kanonické rozhodování přímo do procesu generování. Graf postupně zvětšují pomocí expanzí, ke kterým existují opačné operace nazývané redukce. Pro každý nově vytvořený graf zvolí jednu redukci kanonickým způsobem. Graf přijmou pouze tehdy, pokud právě provedená expanze odpovídá této redukci. Tím lze vytvořit právě jednoho reprezentanta každé izomorfní třídy bez ukládání všech předchozích grafů [1, Section 2.2.1].

Pokud graf obsahuje barvy, musí je kanonická reprezentace také respektovat. Přípustná přeznačení mohou zaměňovat pouze vrcholy se stejnou barvou. Barevné třídy proto musí být součástí kanonického rozhodování již během generování.

1.5 Backtracking a prořezávání

Mnoho generátorů grafů je založeno na backtrackingu. Algoritmus postupně vytváří částečný graf a zkouší všechny přípustné možnosti jeho rozšíření. Pokud

zjistí, že částečný graf nemůže vést k požadovanému výsledku, vrátí se a pokračuje jinou větví výpočtu.

Tomuto předčasnému zahazování větví říkáme prořezávání. Prořezávání může být založeno na jednoduchých podmínkách, například na počtu hran nebo stupních vrcholů, ale také na složitějších algoritmech a heuristikách. Čím dříve dokážeme větev zahodit, tím méně práce musí generátor vykonat (musíme však zaručit, že žádná zahozená větev nemůže vést k přípustnému výslednému grafu, jinak bychom narušili úplnost generování a mohli dojít k chybným závěrům [1, Sections 2.2 and 2.3.2]).

V kontextu této práce budeme rozlišovat filtrování hotových grafů a prořezávání částečně vygenerovaných grafů. Filtrování hotových grafů je jednodušší, protože pracujeme s kompletním grafem. Prořezávání během generování může být výrazně efektivnější, ale musíme počítat s tím, že pracujeme pouze s částečným grafem a některé vlastnosti ještě nemusí být rozhodnutelné.

1.6 Praktické komplikace generování grafů

První praktickou komplikací je kombinatorická exploze. I když generujeme pouze neizomorfní grafy, počet výsledných grafů může být velmi vysoký. Generátory proto musí minimalizovat dobu běhu i spotřebu paměti a často používají kompaktní datové struktury, například bitové množiny [1, Section 2.3.1].

Druhou komplikací je práce s interní reprezentací grafu. Výkonné generátory jsou často psány v jazyce C a jejich vnitřní struktury jsou optimalizovány pro rychlost, nikoli pro pohodlné použití z aplikačního kódu. Pokud chceme nad generovanými grafy spouštět vlastní algoritmy, často musíme psát nízkoúrovňové pluginy nebo pracovat přímo s implementačními detaily generátoru.

Další komplikací je zajištění správnosti. Chyba v prořezávání nebo odmítání izomorfních kopií může způsobit vynechání některých grafů. To je zvláště závažné, pokud chceme na základě úplného generování tvrdit, že graf s určitou vlastností neexistuje. Výsledky proto ověřujeme například porovnáním se známými počty grafů a testováním menších případů [1, Section 2.3.2].

Tyto problémy motivují návrh rozhraní, které zachová výkon specializovaných generátorů a současně poskytne pohodlnější způsob konfigurace, filtrování a zpracování grafů.

Kapitola 2

Generátory grafů geng a plantri

V předchozí kapitole jsme popsali obecný problém generování grafů bez izomorfních kopií. V této kapitole se podíváme na dva konkrétní generátory, na kterých je tato práce postavena: `geng` z balíku `nauty` a `plantri`. Oba nástroje řeší podobný typ úlohy, ale pracují s jinými třídami grafů a používají jiné interní reprezentace.

Pro návrh společného rozhraní potřebujeme vědět, jak se oba generátory konfiguruji, v jaké podobě zpřístupňují aktuální graf a jak umožňují zapojit uživatelský kód do procesu generování.

2.1 Generátor geng

Generátor `geng` je součástí balíku `nauty`. Balík `nauty` je známý především algoritmy pro práci s automorfismy grafů a s kanonickým značením [2]. Program `geng` využívá tyto algoritmy pro generování neizomorfních neorientovaných grafů na zadaném počtu vrcholů. Uživatel může omezit například počet hran, souvislost, minimální a maximální stupeň vrcholů nebo zakázat vybrané malé podgrafy (například trojúhelníky či čtyřcykly).

Takový graf můžeme popsat množinou vrcholů a množinou hran. Při izomorfismu nás tedy zajímá pouze zachování sousednosti. Reprezentace grafu v `nauty` je proto založena na bitových množinách sousedů, které jsou velmi kompaktní a dobře se hodí pro rychlé množinové operace.

`geng` vytváří grafy inkrementálně. Z grafu na n vrcholech vytvoří kandidáty na $n + 1$ vrcholech přidáním nového posledního vrcholu a volbou jeho sousedů mezi původními vrcholy. Ne všechna rozšíření však dále zpracuje. Pomocí automorfismů a kanonického testu odmítá rozšíření, která by vedla k izomorfním kopiím. Při rychlém předběžném rozhodování využívá také jednoduše zjistitelné vlastnosti vrcholů, například jejich stupně.

Z příkazové řádky uživatel zadá počet vrcholů a přepínače určující požado-

vanou třídu grafů. Výsledky může generátor zapisovat například ve formátech graph6 nebo sparse6; výpis lze také vypnout a grafy pouze spočítat.

Při rozšíření geng o barevné třídy a zakotvené vrcholy musí být barvy součástí testů izomorfismu a kanonického rozhodování. Konkrétní změny potřebné pro tuto podporu popisuje kapitola o implementaci.

2.2 Generátor plantri

Generátor plantri je specializovaný nástroj pro generování rovinných grafů. Podporuje například triangulace, kvadrangulace, jednoduché rovinné grafy a triangulace disku. Uživatel může nastavit také omezení souvislosti, minimálního stupně nebo velikosti generovaných grafů

plantri pracuje s grafem společně s jeho rovinným vložením. Nestačí proto znát pouze dvojice vrcholů spojené hranou. Součástí reprezentace je také cyklické pořadí hran kolem každého vrcholu, které určuje stěny vložení.

Generování probíhá rekurzivním zvětšováním malých počátečních grafů pomocí lokálních operací nazývaných expanze. Jejich opačné operace, které graf zmenšují, nazýváme redukce. Pro každou podporovanou třídu jsou expanze navrženy tak, aby zachovávaly požadované vlastnosti grafu a jeho rovinné vložení. Při generování triangulací může expanze například přidat nový vrchol do stěny a spojit jej s vrcholy na jejím okraji.

Stejný výsledný graf může vzniknout několika různými posloupnostmi expanzí. plantri proto používá metodu kanonické konstrukční cesty popsanou v sekce 1.4. Pro každý vytvořený graf určí kanonickou redukci a provedenou expanzi přijme pouze tehdy, pokud je opačnou operací k této redukci. Z více konstrukčních cest vedoucích k izomorfním grafům tak pokračuje pouze jedna [3, Section 1.2].

Interní reprezentace plantri je založena na orientovaných hranách. Každá neorientovaná hrana je reprezentována dvojicí opačně orientovaných hran. Každá z nich obsahuje odkaz na opačnou orientaci a odkazy na předchozí a následující hranu v cyklickém pořadí kolem počátečního vrcholu.

Tato reprezentace umožňuje efektivně procházet okolí vrcholů i hranice stěn. Pro uživatele, který chce nad vygenerovaným grafem spouštět běžné grafové algoritmy, je však práce přímo s orientovanými hranami a ukazateli méně pohodlná. Návrh rozhraní proto musí tuto reprezentaci zpřístupnit bez nutnosti kopírovat celý graf do jiné datové struktury.

2.3 Možnosti rozšíření pomocí callbacků

Oba generátory umožňují zapojit do generování vlastní kód. Původní programy však neposkytují registraci callbacků za běhu. Uživatelské funkce se připojují pomocí maker preprocesoru a generátor je nutné znovu sestavit.

geng poskytuje mechanismy OUTPROC, PRUNE a PREPRUNE. Makro OUTPROC určuje funkci, která se zavolá pro každý výstupní graf namísto standardního zápisu. Uživatel tak může graf přímo zpracovat nebo jej zapsat ve vlastním formátu.

Funkce určená makrem PRUNE se volá nad průběžně vytvářenými i výslednými grafy a může rozhodnout o ukončení aktuální větve generování. PREPRUNE má podobný účel, ale volá se dříve a častěji. Může proto odmítnout nevhodný kandidát před provedením dalších testů generátoru. Test použitý v PREPRUNE by měl být rychlý, protože se během generování vyhodnocuje mnohokrát.

Rozšíření programu plantri se vytvářejí pomocí pluginů. Zdrojový soubor pluginu se při překladu vloží do programu a může definovat několik předem určených maker. Funkce zadaná makrem FILTER se volá pro vygenerované grafy a rozhoduje, zda mají být přijaty. Pro dřívější prořezávání slouží rodina maker PRE_FILTER_*. Konkrétní makro závisí na generované třídě, například na tom, zda generujeme jednoduché rovinné grafy, triangulace nebo kvadrangulace.

Prořezávání během konstrukce může být výrazně účinnější než filtrování hotového výstupu, protože jedním rozhodnutím odstraníme celou část prohledávaného stromu. Musíme však zaručit, že callback neodmítne větev, která by mohla vést k přípustnému výsledku.

Tyto mechanismy jsou výkonné, jejich použití však vyžaduje znalost přesných signatur funkcí, maker použitých při překladu a interní reprezentace grafu.

2.4 Omezení původních rozhraní

Původní rozhraní generátorů geng a plantri jsou dobře přizpůsobena použití z příkazové řádky. Pokud chceme pouze nastavit požadovanou třídu grafů a zapsat výsledky do souboru, jsou oba programy rychlé a praktické. Jejich zapojení do většího programu je však obtížnější.

Každý generátor používá vlastní přepínače, výstupní formáty a způsob konfigurace. Při přímém volání z programu musíme sestavit pole argumentů odpovídající příkazové řádce. Část stavu generování je navíc uložena v globálních proměnných původních C programů.

Dalším omezením je způsob připojení vlastního kódu. Uživatelské funkce jsou určeny již při překladu generátoru pomocí maker nebo pluginů. Jejich změna proto vyžaduje nové sestavení příslušné varianty programu.

Oba generátory také používají rozdílné interní reprezentace grafů. `geng` pracuje s bitovými množinami knihovny `nauty`, zatímco `plantri` používá pole stupňů a ukazatele na orientované hrany. Uživatel musí tyto reprezentace znát, pokud chce v callbacku procházet vrcholy, hrany nebo sousedy. Převod každého grafu do společné datové struktury by práci zjednodušil, ale při velkém počtu grafů by přidával opakovanou režii.

Potřebujeme proto rozhraní, které poskytne společný způsob konfigurace, spouštění a registrace callbacků, ale současně zachová nastavení specifická pro jednotlivé generátory. Grafy by mělo zpřístupňovat bez automatického kopírování jejich interních dat.

Kapitola 3

Návrh rozhraní

V předchozí kapitole jsme popsali možnosti a omezení původních rozhraní generátorů `geng` a `plantri`. V této kapitole představíme návrh knihovny `PlanG`, která oba generátory zpřístupňuje prostřednictvím společného rozhraní v jazyce C++.

Zaměříme se na rozdělení společných a specifických částí rozhraní, registraci callbacků, zpřístupnění grafů bez jejich kopírování a integraci s Boost Graph Library. Technické propojení s původními C programy je popsáno v kapitole o implementaci.

3.1 Cíle návrhu

Původní generátory použijeme jako backendy a nad nimi nabídneme společné C++ rozhraní. Uživatel si zvolí požadovaný backend, nastaví parametry generování pomocí typově bezpečných metod a připojí vlastní funkce pro filtrování, prořezávání nebo zpracování výsledků. Společné operace mají být pro oba generátory stejné, zatímco nastavení závislá na konkrétním generátoru zůstávají součástí jeho backendu.

Důležitým požadavkem je zachování výkonu původních programů. Rozhraní proto nemá automaticky převádět každý vygenerovaný graf do nové datové struktury. Místo toho graf zpřístupní pomocí view nad interními daty generátoru. Uživatel tak může graf zpracovat přímo v callbacku bez vytvoření jeho úplné kopie.

Návrh má také umožnit použití běžných grafových algoritmů a případné doplnění dalších generátorů. Rozhraní proto odděluje společné řízení generování od vlastností konkrétního backendu a přizpůsobuje grafová views vybraným konceptům Boost Graph Library.

3.2 Jednotné rozhraní pro více generátorů

Základním vstupním bodem knihovny je šablona `Generator<Backend>`. Typ předaný jako `Backend` určuje použitý původní generátor a dostupná specifická nastavení. Pro obecné grafy uživatel vytváří objekt typu `Generator<geng::Backend>`, zatímco pro rovinné grafy objekt typu `Generator<plantri::Backend>`.

Šablona `Generator` poskytuje společné operace, zejména spuštění generování metodou `run` a registraci callbacků. Zvolený backend doplňuje metody pro konfiguraci vlastností podporovaných konkrétním generátorem. Uživatel tak pracuje s oběma generátory stejným způsobem, ale zároveň má přístup k jejich specifickým možnostem.

Toto uspořádání jsme založili na návrhu využívajícím politiky (policy-based design). Konkrétní backend je zvolen jako parametr šablony a poskytuje třídě `Generator` chování závislé na daném generátoru. Volba backendu tak probíhá při překladu a nevyžaduje virtuální funkce.

Vývojář, který chce do PlanG přidat další generátor, musí vytvořit backend poskytující typ grafového view, vlastní konfiguraci a operace potřebné ke spuštění generátoru. Samotný způsob, jakým uživatel spouští generování nebo registruje callbacky, se přidáním backendu nemění.

3.3 Callbacky pro filtrování a zpracování grafů

PlanG poskytuje tři typy callbacků, které odpovídají různým fázím generování. Uživatel je registruje jako C++ funkční objekty před spuštěním metody `run`. Jako callback může předat běžnou funkci, lambda výraz nebo objekt s operátorem volání.

Metodou `setFilter` uživatel registruje filtr výsledných grafů. PlanG jej volá pro každý dokončený graf. Callback vrátí hodnotu `KEEP`, pokud má být graf ponechán, nebo `PRUNE`, pokud má být odmítnut.

Metodou `setPreprune` uživatel registruje callback pro prořezávání rozpracovaných větví. Callback rozhoduje, zda má původní generátor pokračovat v aktuální větvi. Generátor jej může volat mnohem častěji než filtr výsledných grafů, proto by měl prováděný test být rychlý a nesmí odmítnout větev, která by mohla vést k přípustnému výsledku.

Metodou `setOutproc` uživatel registruje funkci pro zpracování přijatých výsledných grafů. Může ji využít pro vlastní výstup, sběr statistik nebo spuštění dalšího grafového algoritmu.

PlanG předává uživatelským callbackům objekt `GraphView`, který zpřístupňuje aktuální graf bez vytvoření jeho úplné kopie. Uživatelský kód tak pracuje s grafem stejným způsobem nezávisle na zvoleném backendu.

3.4 View na graf bez kopírování dat

Abychom omezili režii při zpracování velkého počtu grafů, rozhodli jsme se nepřevádět každý výsledek do nové vlastněné datové struktury. Pro každý backend jsme proto definovali samostatný typ `GraphView`, který zpřístupňuje graf uložený uvnitř původního generátoru.

`geng` : `GraphView` uchovává ukazatel na bitovou reprezentaci nauty, zatímco `plantri` : `GraphView` pracuje s poli stupňů a orientovanými hranami rovinného vložení. Při vytvoření objektu `GraphView PlanG` kopíruje pouze několik ukazatelů a základních údajů o grafu, nikoli jeho vrcholy a hrany.

Uživatel uvnitř callbacku pracuje s grafem zpravidla prostřednictvím globálních funkcí, které přijímají graf jako parametr. Nejde tedy o členské metody `GraphView`. Mezi tyto funkce patří například `vertices(g)`, `edges(g)`, `adjacent_vertices(v, g)` nebo `out_edges(v, g)`. Tyto funkce tvoří rozhraní požadované vybranými koncepty Boost Graph Library a jejich implementaci poskytuje `PlanG`. Uživatel je může volat přímo nebo předat objekt `GraphView` kompatibilnímu algoritmu z této knihovny.

3.5 Integrace s Boost Graph Library

Boost Graph Library je založena na grafových konceptech, které určují, jaké typy a operace musí graf poskytovat, aby nad ním bylo možné spouštět obecné grafové algoritmy [4]. Graf přitom nemusí být instancí konkrétní třídy z knihovny Boost, pokud splňuje požadované rozhraní.

Pro oba typy `GraphView` jsme definovali typy vrcholů a hran, iterátory a podporované způsoby procházení grafu. Společně s globálními funkcemi popsány v předchozí sekci tak oba typy splňují vybrané koncepty Boost Graph Library. Uživatel tyto adaptéry nemusí sám doplňovat.

Díky této integraci může uživatel v callbacku spouštět například algoritmy pro barvení vrcholů nebo testování rovinnosti. Konkrétní implementace potřebných typů, iterátorů a traits je popsána v kapitole o implementaci.

3.6 Podpora bohatších struktur

Veřejné rozhraní backendu `geng` podporuje generování grafů s barevnými třídami vrcholů. Uživatel může nastavit počet barev, přesné velikosti tříd nebo jejich dolní a horní meze pomocí metod `setColors`, `setColorClassSizes` a `setColorClassBounds`.

Uživatel zadává počet zakotvených vrcholů metodou `setRootedVertices`.

Backend interně reprezentuje každý zakotvený vrchol samostatnou jednoprvkovou barevnou třídou, kterou musí izomorfismus zachovat.

PlanG zpřístupňuje přiřazené barvy také prostřednictvím `geng::GraphView`, aby s nimi mohly pracovat uživatelské callbacky. Způsob generování barev a jejich zahrnutí do kanonických testů je popsán v kapitole o implementaci.

Kapitola 4

Implementace

V této kapitole popíšeme implementaci knihovny PlanG. V předchozí kapitole jsme navrhli rozhraní, které má sjednotit práci s generátory geng a plantri. Nyní se podíváme na to, jak je toto rozhraní napojeno na původní C programy.

C++ část knihovny převádí konfiguraci uloženou v objektu na argumenty původního programu. Callbacky a konfiguraci obarvení, které nejsou součástí argumentů příkazové řádky, předává před spuštěním generátoru do C mezivrstvy. Aktuální graf následně zpřístupňuje pomocí view nad interními daty generátoru.

4.1 Architektura knihovny

Implementaci jsme rozdělili do několika vrstev. Nejnižší vrstvu tvoří zdrojové kódy původních generátorů. Jejich algoritmy i způsob zpracování argumentů příkazové řádky jsme zachovali. Výjimkou jsou úpravy generátoru geng, kterými jsme doplnili podporu barevných tříd a zakotvených vrcholů.

Nad původními generátory se nachází tenká C mezivrstva. Ta zpřístupňuje jejich vstupní funkce a vybrané části globálního stavu. Současně obsahuje funkce, na které jsou přesměrována původní makra pro filtrování, prořezávání a zpracování výstupu.

Další vrstvu tvoří C++ bridge. Uchovává uživatelské callbacky, převádí volání z C mezivrstvy na C++ funkční objekty a vytváří pro ně příslušný GraphView. Zajišťuje také, aby výjimky vyvolané callbackem neprocházely přes C kód.

Nad bridge vrstvou jsme implementovali backendy `geng::Backend` a `plantri::Backend`. Každý backend uchovává konfiguraci jedné instance, ověřuje její správnost a převádí ji na argumenty odpovídajícího generátoru. Před spuštěním nastavuje také callbacky a další stav, který nelze předat argumenty příkazové řádky.

Nejvyšší vrstvu tvoří šablona `Generator<Backend>`. Poskytuje společný po-

stup spuštění a zpřístupňuje veřejné metody zvoleného backendu. Tok volání tak vede od objektu `Generator` přes backend a mezivrstvy až k původnímu generátoru. Při vyvolání callbacku původním generátorem prochází volání mezivrstvami opačným směrem do uživatelského C++ kódu.

4.2 Vrstva `Generator` a backendy

Šablona `Generator<Backend>` veřejně dědí ze zvoleného backendu. Uživatel proto může na objektu generátoru přímo volat konfigurační metody backendu. Společná vrstva přidává metodu `run`, typovou zkratku `GraphView` a metody pro registraci callbacků.

Při volání metody `run` uživatelským programem šablona `Generator` nejprve volá metodu `prepare_args` příslušného backendu. Backend ověřuje uloženou konfiguraci a vytváří argumenty ve stejném tvaru, v jakém je očekává původní program. Tím jsme zachovali existující zpracování argumentů v `geng` a `plantri`.

Šablona `Generator` následně volá metodu `apply_runtime_state`. Backend jejím prostřednictvím předává do bridge vrstvy uživatelské callbacky a nastavení, která nelze vyjádřit argumenty příkazové řádky. U backendu `geng` sem patří také konfigurace barevných tříd.

Nakonec šablona `Generator` převádí vytvořené argumenty do formátu `argc/argv` očekávaného původním C programem a předává je metodě `Backend::run`, která spouští odpovídající generátor.

Každá instance backendu uchovává vlastní konfiguraci, takže uživatelský program může v jednom procesu postupně spouštět více instancí s rozdílným nastavením. Původní generátory a bridge vrstva však používají globální stav nastavovaný před spuštěním. `PlanG` proto nepodporuje současné paralelní běhy stejného backendu.

4.3 Propojení s `geng`

Backend `geng::Backend` uchovává konfiguraci odpovídající přepínačům původního programu. Patří sem například počet vrcholů, rozsah počtu hran, omezení stupňů, požadovaná souvislost, výstupní formát a rozdělení výpočtu. Metoda `prepare_args` z těchto hodnot sestavuje argumenty pro `geng`.

Při sestavení knihovny jsme původní funkci `main` generátoru přejmenovali na `geng_main` a makra `OUTPROC`, `PRUNE` a `PREPRUNE` jsme přesměrovali do C mezivrstvy. Tyto úpravy jsou součástí sestavení `PlanG`; uživatelský program pouze odkazuje na výslednou knihovnu.

C mezivrstva uchovává ukazatele na funkce registrované z C++ části. Při volání některého z rozšiřujících mechanismů generátorem geng předává mezivrstva aktuální graf odpovídající trampoline funkci. Ta vytváří `geng::GraphView` obsahující ukazatel na bitovou reprezentaci nauty, aktuální počet vrcholů, cílový počet vrcholů a případné barvy.

Původní mechanismus PRUNE se volá nad mezilehlými i výslednými grafy. PlanG však callback registrovaný metodou `setFilter` volá pouze pro hotové grafy. Trampoline funkce proto nejprve porovnává aktuální a cílový počet vrcholů. Pokud se liší, uživatelský callback nevolá. Pro prořezávání mezilehlých grafů může uživatel registrovat samostatný callback metodou `setPreprune`, kterou PlanG napojuje na mechanismus PREPRUNE.

Při čistém generování bez callbacku pro zpracování výstupu PlanG zachovává původní výstupní funkce programu geng. Přejechod přes `OUTPROC` aktivuje pouze tehdy, když uživatel nastavil `setOutproc` nebo zapnul generování barevných grafů. Při běžném generování se tak mezivrstva nevolá pro každý nalezený graf.

4.4 Propojení s plantri

Backend `plantri::Backend` uchovává například třídu generovaných grafů, počet vrcholů, minimální stupeň, požadovanou souvislost, výstupní formát a parametry rozdělení výpočtu. Metoda `prepare_args` tato nastavení převádí na argumenty původního programu.

Při sestavení knihovny jsme původní funkci `main` přejmenovali na `plantri_run` a C mezivrstvu jsme připojili pomocí pluginového mechanismu `plantri`. Její zdrojový kód je přitom vložen přímo do stejné překladové jednotky a mezivrstva může přistupovat k interním funkcím a globálním proměnným generátoru. Tyto kroky probíhají při sestavení PlanG, nikoli v uživatelském programu.

Makro `FILTER` jsme přesměrovali na funkci C mezivrstvy obsluhující `setFilter`. Běžné varianty `maker PRE_FILTER_*` jsme přesměrovali na společnou funkci obsluhující `setPreprune`. PlanG tak skrývá rozdíly mezi variantami `maker` používanými pro různé třídy grafů.

PlanG obsluhuje callback registrovaný metodou `setFilter` prostřednictvím mechanismu `FILTER`. Stejnou cestu používá také pro `setOutproc`. Trampoline funkce nejprve volá uživatelský filtr a při přijetí grafu jej případně předává callbacku pro zpracování výstupu. Pokud uživatel nastavil `setOutproc`, program `plantri` již standardní zápis neprovádí.

Protože `plantri` drží aktuální graf v globálních proměnných, C mezivrstva zpřístupňuje funkce pro čtení těchto hodnot. C++ vrstva z nich sestavuje view pro backend `plantri`. Do něj se předávají ukazatele na hrany, pole stupňů vrcholů, počet vrcholů, počet orientovaných hran a cílový počet vrcholů.

4.5 Integrace s Boost Graph Library

Boost Graph Library používá šablonu `boost::graph_traits`¹, pomocí které získává typy vrcholů, hran a iterátorů grafu. Potřebné typy jsme definovali přímo v obou třídách `GraphView`. Tyto adaptéry jsou součástí `PlanG` a uživatel je nemusí sám implementovat. Oba backendy podporují koncepty `VertexListGraph`, `IncidenceGraph` a `AdjacencyGraph`.

Vrcholy jsme v obou backendech reprezentovali typem `int`, protože původní generátory již používají celočíselné indexy. Pro `geng` jsme hranu reprezentovali dvojicí koncových vrcholů. Generované grafy jsou jednoduché a interní bitová reprezentace neobsahuje samostatné objekty hran.

U `plantri` jsme jako descriptor hrany použili obal nad ukazatelem `EDGE*`. Samotná dvojice koncových vrcholů by nestačila, protože `plantri` pracuje s orientovanými hranami a může obsahovat paralelní hrany. Ukazatel zachovává identitu konkrétní hrany i přístup k její opačné orientaci a k sousedním hranám v rovinném vložení.

Iterátory jsme implementovali pomocí `boost::iterator_facade`. U backendu `geng` iterátory hledají sousedy přímo v bitových množinách. U backendu `plantri` iterátory sledují odkazy `next` mezi orientovanými hranami. Při procházení všech hran vrací pouze jednu orientaci každé neorientované hrany.

Požadované operace jsme implementovali jako globální funkce v jmenném prostoru příslušného backendu, které přijímají graf jako parametr. Patří mezi ně například `vertices(g)`, `edges(g)`, `out_edges(v, g)`, `adjacent_vertices(v, g)`, `source(e, g)` a `target(e, g)`. Pro indexování vrcholů používá `geng` identické mapování. U `plantri` vlastní mapa indexů přeskakuje případnou hodnotu `missing_vertex`, aby algoritmy pracovaly se souvislými indexy od nuly.

4.6 Obarvené a zakotvené vrcholy v `geng`

Podporu barevných tříd jsme implementovali přímo v původním algoritmu `geng`. Upravený generátor pro každou barvu uchovává dolní a horní mez velikosti třídy, aktuální počet jejích vrcholů a barvu každého vrcholu rozpracovaného grafu.

Při přidávání nového vrcholu zkouší původní algoritmus přípustné množiny jeho sousedů. Při aktivním obarvení upravený algoritmus pro každou z nich navíc zkouší všechny barvy nového vrcholu. Zjednodušený průběh ukazuje výpis 1.

Před provedením kanonického testu upravený `geng` ověřuje, že žádná barevná třída nepřekročila horní mez a že zbývající vrcholy lze rozdělit tak, aby byly splněny všechny zadané meze. Pokud tato podmínka neplatí, generátor přiřazení barvy ihned odmítá.

¹https://www.boost.org/doc/libs/latest/libs/graph/doc/graph_traits.html

Výpis kódu 1 Zjednodušené rozšíření grafu o obarvený vrchol.

```
extend(graf G na n vrcholech):  
    pro každou přípustnou množinu sousedů nového vrcholu:  
        pokud obarvení není aktivní:  
            pokračuj původním algoritmem geng  
  
        pokud obarvení je aktivní:  
            pro každou barvu c:  
                přiřaď novému vrcholu barvu c  
                zvyš aktuální počet vrcholů v barvě c  
  
            pokud lze ještě splnit dolní a horní meze barev:  
                proveď accept test s partition rozdělenou podle barev  
                pokud test projde:  
                    pokračuj rekurzivně  
  
    vrať přiřazení barvy zpět
```

Aby kanonický test respektoval barvy, upravený generátor dělí každou buňku počáteční partition dále podle barev vrcholů. Vrcholy různých barev jsou tím zařazeny do různých buněk a nauty je při hledání automorfismů nemůže vzájemně zaměnit.

Rozdělení podle barev může vést k situaci, kdy každý vrchol tvoří samostatnou buňku ještě před zahájením refinementu. Původní implementace kontrolovala pozici posledního přidaného vrcholu až během dalšího dělení buněk, takže se tato kontrola v uvedeném případě neprovedla. Doplnili jsme proto zvláštní kontrolu pro úplné rozdělení. Pokud poslední přidaný vrchol neleží na poslední pozici výsledného uspořádání, nová kontrola rozšíření odmítá.

Podporu obarvení jsme doplnili do standardní (genextend) i specializovaných (spaextend) větví generování, aby fungovala také při použití dalších omezení geng. Pokud obarvení není aktivní, upravený generátor pokračuje původní cestou algoritmu bez zkoušení barev.

Zakotvené vrcholy jsme implementovali pomocí samostatných barevných tříd velikosti jedna. Uživatel zadává jejich počet metodou `setRootedVertices` a backend je interně převádí na odpovídající barevné třídy. Izomorfismus proto nemůže zakotvený vrchol zaměnit s ostatními vrcholy ani s jiným zakotveným vrcholem. Při kanonickém přeznačení grafu upravený generátor přeznačuje stejnou permutací také pole barev, aby přiřazené barvy odpovídaly výslednému označení vrcholů.

Kapitola 5

Testování a vyhodnocení

V této kapitole jsme ověřili, že knihovna PlanG zachovává chování původních generátorů, a porovnali jsme její rychlost s nativními programy. Měření se zaměřilo na režii C++ rozhraní, volání callbacků a generování grafů s dodatečnou strukturou.

5.1 Funkční testování

Nejdříve jsme ověřili, že rozhraní pro malé instance vytváří stejný výstup jako původní programy. Pro geng jsme porovnali výstup ve formátu graph6 pro grafy na šesti vrcholech. Pro plantri jsme stejným způsobem porovnali triangulace na osmi vrcholech vypsane také ve formátu graph6. První porovnání obsahovalo 156 grafů a druhé 14 triangulací. V obou případech se soubory vytvořené přes PlanG shodovaly se soubory vytvořenými nativním generátorem.

Dále jsme testovali běhy s callbacky `setFilter`, `setPreprune` a `setOutproc`. Testy ověřily, že PlanG volá callbacky ve správné fázi generování a prostřednictvím `GraphView` jim zpřístupňuje očekávaný aktuální graf. Pro rozšíření geng o barevné třídy jsme testovali také přesné velikosti barevných tříd, intervalové meze tříd a zakotvené vrcholy. Testy byly provedeny na vybraných konfiguracích a ověřují správnost testovaných případů.

5.2 Metodika měření

Měření probíhalo na počítači s procesorem Intel Core i9-13900H v prostředí Linux/WSL2. Pro porovnání jsme použili geng z balíku nauty 2.9.1 a plantri verze 5.5. Nativní programy i varianty integrované v PlanG jsme sestavili se stejnými optimalizačními přepínači `-O3 -flto -march=native -DNDEBUG`.

Aby měření nebylo ovlivněno rychlostí zápisu na výstup, vypnuli jsme vypisování grafů. U nativních programů jsme použili přepínač `-u`; u `geng` navíc přepínač `-q`, který potlačí pomocný výstup. V `PlanG` jsme použili odpovídající metodu `setNoOutput`. Každý běh jsme spustili jako samostatný proces a pomocí příkazu `time` jsme měřili jeho celkový čas `real`. Jednotlivé konfigurace jsme spustili alespoň třikrát a v tabulkách uvádíme medián naměřených časů. Poslední sloupec s poměrem časů udává čas `PlanG` dělený časem nativního programu; hodnota jedna tedy odpovídá stejné době běhu.

5.3 Čisté generování

Nejprve jsme porovnali běh bez uživatelských callbacků. V tomto režimu C++ část knihovny pouze převádí konfiguraci na argumenty původního programu, nastavuje prázdný runtime stav a spouští původní generátor. Měření tak zachycuje základní režii použití rozhraní `PlanG`.

n	počet grafů	<code>geng</code> [s]	<code>PlanG</code> [s]	<code>PlanG/geng</code>
8	12 346	0,008	0,009	1,13
9	274 668	0,068	0,067	0,99
10	12 005 168	2,101	2,169	1,03
11	1 018 997 864	237,542	243,459	1,02

Tabulka 5.1 Porovnání nativního `geng` a `PlanG` při generování bez výstupu.

Výsledky v tabulce 5.1 ukazují, že použití rozhraní `PlanG` nevedlo při čistém generování k výraznému zpomalení. Výkon obou variant tak zůstává srovnatelný.

n	počet triangulací	<code>plantri</code> [s]	<code>PlanG</code> [s]	<code>PlanG/plantri</code>
14	339 722	0,125	0,123	0,98
15	2 406 841	0,869	0,838	0,96
16	17 490 241	6,169	6,080	0,99

Tabulka 5.2 Porovnání nativního `plantri` a `PlanG` při generování bez výstupu.

Také u `plantri` zůstává běh přes `PlanG` srovnatelný s nativním programem, jak ukazuje tabulka 5.2.

5.4 Callbacky a srovnání s pluginem

Nejprve jsme změřili samotnou režii callbacků v backendu geng. Použité callbacky neprováděly žádný výpočet a vždy vracely hodnotu KEEP, takže neměnily průběh ani výsledek generování. Metoda `setFilter` registrovala callback volaný nad dokončenými grafy, zatímco callback registrovaný metodou `setPreprune` se volal také během generování rozpracovaných grafů.

n	bez callbacku [s]	<code>setFilter</code> [s]	<code>setPreprune</code> [s]
9	0,078	0,079	0,074
10	2,408	2,417	2,463

Tabulka 5.3 Režie prázdných callbacků v backendu geng.

Z výsledků v tabulce 5.3 vyplývá, že prázdné callbacky nepřidávají významnou režii. Pro $n = 10$ zůstává rozdíl oproti běhu bez callbacku velmi malý. Callback `setPreprune` je o něco dražší, protože se volá i nad rozpracovanými grafy.

Pro praktičtější porovnání jsme použili plugin `maxdeg.c` dodávaný s `plantri`. Plugin během generování triangulací prořezává větve, které nemohou vést ke grafu s požadovaným maximálním stupněm. Nativní variantu jsme sestavili přímo s pluginem. V PlanG jsme stejný test zapsali pomocí `setPreprune` a výslednou podmínku ověřili pomocí `setFilter`. V obou variantách jsme nastavili maximální stupeň sedm.¹

n	počet triangulací	<code>plantri_maxd</code> [s]	PlanG [s]	PlanG/native
16	193 445	0,295	0,306	1,04
17	686 364	1,120	1,157	1,03
18	2 444 816	4,366	4,465	1,02

Tabulka 5.4 Porovnání nativního pluginu `maxdeg.c` a callbackové implementace v PlanG.

Obě implementace vytvořily pro každou hodnotu n stejný počet triangulací. Výsledky v tabulce 5.4 ukazují, že callbacková implementace v PlanG byla jen mírně pomalejší než nativní plugin. Zároveň je vidět, že relativní rozdíl mezi oběma variantami se pro větší hodnoty n zmenšuje.

¹Použité benchmarky jsou dostupné ve složce `benchmarks` projektu.

5.5 Cena bohatších struktur

Rozšíření geng o barevné třídy a zakotvené vrcholy nelze přímo porovnat s nativním geng, protože původní program tyto struktury nepodporuje. Přesto je užitečné změřit, jakou cenu tato podpora přináší.

n	bez barev [s]	dvě vyvážené barvy [s]	dva zakotvené vrcholy [s]
8	0,007	0,121	0,108
9	0,070	4,958	3,498

Tabulka 5.5 Čas generování geng v PlanG při zapnutí barevných tříd.

Tabulka 5.5 neměří pouze režii API. Po zapnutí barev řeší generátor jinou úlohu: při rozšiřování grafu zkouší také barvu nového vrcholu, kontroluje meze velikostí barevných tříd a provádí kanonické testy s partition rozdělenou podle barev. Výrazné zpomalení je tedy očekávané. Na druhou stranu tím získáme možnost generovat grafy s dodatečnou strukturou přímo během kanonického generování, nikoli až následným filtrováním hotových neobarvených grafů.

5.6 Shrnutí

Funkční testy pro vybrané malé instance poskytly stejné výstupy jako původní generátory. Při čistém generování dosáhl PlanG výkonu srovnatelného s nativními programy. Měření prázdných callbacků ukázalo, že samotné callbackové rozhraní nepřidává významnou režii.

Praktické porovnání s pluginem `maxdeg.c` ukázalo, že callbacková implementace v PlanG zůstává výkonově blízko nativnímu pluginu. Uživatel přitom nemusí upravovat původní generátor ani sestavovat vlastní variantu `plantri`; filtr zapisuje jako běžný C++ callback nad objektem `GraphView`.

Generování barevných a zakotvených grafů bylo výrazně náročnější, protože odpovídá obecnější generační úloze s větším množstvím navzájem rozlišitelných struktur. Výsledky celkově ukazují, že PlanG zachovává výkon původních generátorů v základním režimu a zároveň umožňuje pohodlnější filtrování, zpracování a rozšíření generování bez automatického kopírování každého grafu.

Závěr

V této práci jsme se zabývali zpřístupněním generátorů geng a plantri z moderního C++ kódu. Navrhli a implementovali jsme knihovnu PlanG, která nad těmito nástroji poskytuje jednotné rozhraní pro konfiguraci generování, filtrování a zpracování grafů.

Součástí řešení je GraphView, které zpřístupňuje aktuální graf bez automatického kopírování do nové datové struktury. Pro oba backendy jsme připravili rozhraní použitelné s vybranými koncepty Boost Graph Library. Generátor geng jsme navíc rozšířili o podporu barevných tříd a zakotvených vrcholů.

Výsledné řešení jsme ověřili funkčními testy a porovnali s původními generátory. Měření ukázala, že při čistém generování dosahuje PlanG výkonu srovnatelného s nativními programy. PlanG tak umožňuje používat výkonné generátory grafů pohodlněji z C++ programu, aniž by bylo nutné každý graf automaticky kopírovat nebo psát samostatné pluginy.

Možná rozšíření

Dalším směrem rozvoje je podpora dalších generátorů grafů a rozšíření rozhraní o další globální funkce a algoritmy, které pracují přímo nad objekty GraphView.

Seznam použité literatury

- [1] Jorik Jooken. *Computer-assisted graph theory: a survey*. 2025. URL: <https://arxiv.org/pdf/2508.20825>.
- [2] Brendan D. McKay a Adolfo Piperno. *nauty and Traces*. URL: <https://pallini.di.uniroma1.it/>.
- [3] McKay Brinkmann Gunnar a Brendan D. “Fast generation of planar graphs”. In: (). URL: <https://users.cecs.anu.edu.au/~bdm/papers/plantri-full.pdf>.
- [4] Boost C++ Libraries. *Boost Graph Library: Graph Concepts*. URL: https://www.boost.org/doc/libs/latest/libs/graph/doc/graph_concepts.html.

Příloha A

Použití knihovny PlanG

Tato příloha stručně popisuje, jak sestavit projekt PlanG a jak použít jeho nejdůležitější části v C++ programu. Úplnější přehled metod je uveden v souboru `README.md`; programátorská dokumentace v souboru `DEVELOPER.md` popisuje vnitřní návrh knihovny.

A.1 Sestavení projektu

Projekt vyžaduje překladač C++, nástroj `make` a knihovnu Boost. Lze jej sestavit na Linuxu a macOS; na Windows je doporučeno použít WSL. Na Linuxu se používá `gcc` a `g++`, zatímco na macOS instalační skript použije `clang` a `clang++`. Projekt lze sestavit instalačním skriptem `install.sh`:

```
./install.sh
```

Skript nejprve hledá hlavičkové soubory Boostu v běžných umístěních. Pokud je nenajde, pokusí se Boost automaticky doinstalovat: na Linuxu pomocí `apt-get` a na macOS pomocí Homebrew. Poté sestaví oba backendy, zkopíruje statické knihovny do adresáře `lib/` a vytvoří spustitelný soubor `plang`. Pokud je knihovna Boost v nestandardním umístění, lze její cestu předat proměnnou `BOOST_DIR`:

```
BOOST_DIR=/path/to/boost ./install.sh
```

Výchozí program se spustí příkazem:

```
./plang
```

Soubor `main.cpp` slouží jako hlavní ukázkový program. Po jeho změně je potřeba projekt znovu sestavit.

Výpis kódu 2 Jednoduché použití backendu geng.

```
#include "MyGraphLib.hpp"

using App = Generator<geng::Backend>;

int main()
{
    App app;
    app.setVertices(10);
    app.setConnected();
    app.setTriangleFree();
    return app.run();
}
```

Výpis kódu 3 Jednoduché použití backendu plantri.

```
#include "MyGraphLib.hpp"

using App = Generator<plantri::Backend>;

int main()
{
    App app;
    app.setVertices(12);
    app.setFormat(App::OutputFormat::Graph6);
    return app.run();
}
```

A.2 Základní program

Veřejné API knihovny zpřístupňuje hlavičkový soubor `MyGraphLib.hpp`. Uživatel zvolí backend pomocí šablony `Generator<Backend>`. Backend `geng::Backend` používá generátor `geng`, zatímco backend `plantri::Backend` používá generátor `plantri`.

Stejný tvar programu lze použít i pro `plantri`. Liší se pouze zvolený backend a metody dostupné pro konkrétní generátor.

Oba backendy umožňují vypnout výstup metodou `setNoOutput` nebo zapsat výsledek do souboru metodou `setOutputFile`. Backend `geng` podporuje například omezení počtu hran, souvislost, bipartitnost a stupňová omezení. Backend `plantri` podporuje volbu třídy rovinných grafů, minimální stupeň, konektivitu, výstup duálního grafu a různé výstupní formáty.

Výpis kódu 4 Filtrování dokončených grafů pomocí `setFilter`.

```
#include "MyGraphLib.hpp"

using App = Generator<geng::Backend>;

int main()
{
    App app;
    app.setVertices(6);

    app.setFilter([](const App::GraphView& g) {
        return num_edges(g) > 0 ? KEEP : PRUNE;
    });

    return app.run();
}
```

A.3 Callbacky

Knihovna nabízí tři základní callbacky. Callback `setFilter` se používá pro filtrování dokončených grafů. Callback `setPreprune` se volá během generování a může předčasně zahodit rozpracovanou větev. Callback `setOutproc` slouží k vlastnímu zpracování grafu, který prošel filtrem.

Filtreační callbacky vracejí hodnotu `KEEP`, pokud má graf nebo větev zůstat zachována, a hodnotu `PRUNE`, pokud má být odmítnuta. Následující program generuje grafy pomocí `geng` a ponechá pouze grafy, které mají alespoň jednu hranu.

Callback `setOutproc` se hodí tehdy, když program nemá grafy pouze vypsat v původním formátu, ale chce nad nimi provést vlastní výpočet. V následujícím příkladu program pouze spočítá, kolik grafů generátor vytvořil.

A.4 Práce s grafem

Callbacky dostávají objekt `GraphView`. Tento objekt je pohled na graf uložený uvnitř původního generátoru. Graf tedy není automaticky kopírován do nové datové struktury. Objekt `GraphView` je určený pro použití uvnitř callbacku; po pokračování generování se interní data mohou změnit.

Pro práci s grafem poskytuje PlanG globální funkce, které berou graf jako parametr. Patří mezi ně například `vertices(g)`, `edges(g)`, `num_vertices(g)`, `num_edges(g)`, `out_edges(v, g)`, `adjacent_vertices(v, g)`, `source(e, g)` a `target(e, g)`. Tyto funkce lze používat přímo v uživatelském callbacku.

Výpis kódu 5 Vlastní zpracování výstupu pomocí `setOutproc`.

```
#include "MyGraphLib.hpp"
#include <iostream>

using App = Generator<plantri::Backend>;

int main()
{
    App app;
    int count = 0;

    app.setVertices(10);
    app.setNoOutput();

    app.setOutproc([&](Output&, const App::GraphView&) {
        ++count;
    });

    int result = app.run();
    std::cout << count << "\n";
    return result;
}
```

Výpis kódu 6 Použití globálních funkcí pro práci s objektem `GraphView`.

```
app.setFilter([](const App::GraphView& g) {
    auto [first, last] = vertices(g);

    for (auto it = first; it != last; ++it)
    {
        int v = *it;
        if (out_degree(v, g) == 0)
            return PRUNE;
    }

    return KEEP;
});
```

Výpis kódu 7 Použití algoritmu z Boost Graph Library nad objektem GraphView.

```
#include "MyGraphLib.hpp"
#include <boost/graph/sequential_vertex_coloring.hpp>
#include <vector>

using App = Generator<plantri::Backend>;

int main()
{
    App app;
    app.setVertices(13);

    app.setFilter([](const App::GraphView& g) {
        std::vector<int> colors(num_vertices(g));
        int used = boost::sequential_vertex_coloring(g, colors.data());
        return used >= 4 ? KEEP : PRUNE;
    });

    return app.run();
}
```

U backendu plantri jsou navíc dostupné funkce pro práci s orientovanými hranami rovinného vložení, například `next_edge`, `prev_edge` a `opposite_edge`. Tyto funkce jsou užitečné u algoritmů, které potřebují pracovat přímo s cyklickým pořadím hran kolem vrcholů.

A.5 Použití algoritmů Boost Graph Library

Objekty GraphView jsou přizpůsobeny vybraným konceptům Boost Graph Library. Uživatel proto může v callbacku spustit standardní grafový algoritmus, aniž by graf převáděl do datové struktury z Boostu. Následující příklad používá algoritmus `boost::sequential_vertex_coloring`.

A.6 Obarvené a zakotvené vrcholy v backendu geng

Backend geng podporuje také generování grafů s barevnými třídami vrcholů. Přesné velikosti barevných tříd se nastaví metodou `setColorClassSizes`. Pokud stačí zadat pouze počet barev, lze použít `setColors`. Zakotvené vrcholy se nastavují metodou `setRootedVertices`.

Výpis kódu 8 Generování grafů s dvěma barevnými třídami.

```
#include "MyGraphLib.hpp"

using App = Generator<geng::Backend>;

int main()
{
    App app;
    app.setVertices(5);
    app.setColorClassSizes({3, 2});
    app.setCanonicalLabeling();

    app.setFilter([](const App::GraphView& g) {
        int cross_edges = 0;

        auto [first, last] = edges(g);
        for (auto it = first; it != last; ++it)
        {
            auto [u, v] = *it;
            if (g.color(u) != g.color(v))
                ++cross_edges;
        }

        return cross_edges >= 2 ? KEEP : PRUNE;
    });

    return app.run();
}
```

Barvy lze v callbacku číst přímo z objektu `GraphView`. K dispozici jsou metody `has_coloring()`, `color_count()`, `color(v)`, `vertex_colors()` a `vertices_of_color(c)`. Zakotvené vrcholy jsou reprezentovány jako samostatné jednoprvkové barevné třídy.

A.7 Dostupné ukázky a benchmarky

Adresář `examples/` obsahuje další ukázkové programy demonstrující použití knihovny. Adresář `benchmarks/` obsahuje programy použité pro měření v této práci.

Zdrojové kódy projektu jsou dostupné v repozitáři: <https://github.com/dochynes/PlanG/tree/main>

